Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică
Program: Software Engineering

Operating Systems

Laboratory Work No. 4

The xv6 Shell from the Outside

Track B: Build Your Own Small OS (xv6)

Done by: st. gr. FAF-241, Ermicev Sofia

Checked by: Patricia Reițman

Chișinău – 2026

Contents

1 Purpose of the Work **2**

2 Part 1. Fixing and Testing sleep **2**

2.1 Observation 2

3 Part 2. Bash versus the xv6 Shell **2**

3.1 Observe answers 3

4 Part 3. Who Expands *? Inspecting argc and argv **3**

4.1 Observe answers 4

5 Part 4. Reading the xv6 Shell Source **4**

5.1 Observe answers 5

6 Conclusion **5**

1. Purpose of the Work

The purpose of this laboratory work is to compare the xv6 shell with Bash, adapt a small program to the current xv6 API, inspect argument handling, and connect the observed behavior with the implementation in `user/sh.c`.

2. Part 1. Fixing and Testing sleep

```
25:int pause(int);
```

Figure 1.1 – Actual current xv6 user API entry for pause

The current xv6 tree uses `pause(int)` rather than the older `sleep(int)` system call. The repository version of `user/sleep.c` therefore calls `pause(atoi(argv[1`

```
$ sleep 20
$ sleep
usage: sleep <ticks>
```

Figure 1.2 – Actual sleep tests in xv6

2.1. Observation

With one argument the program waits for the requested number of ticks. With a missing argument it prints the usage message and exits with an error.

3. Part 2. Bash versus the xv6 Shell

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
bg
```

```
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
```

Figure 2.1 – Actual selected xv6-shell results

```
$ cd d
$ ls
exec ls failed
```

Figure 2.2 – Actual failure of PATH-style command lookup after cd in xv6

3.1. Observe answers

1. Both shells support normal program execution, redirection, pipes, lists, grouped commands and background execution. Bash additionally provides globbing, variable expansion, quote processing, PATH lookup, history, completion and richer job control.
2. After `cd d`, xv6 cannot find `ls` by name because it does not search PATH. The executable can be addressed explicitly from the parent directory.
3. PATH lookup would be one of the more straightforward conveniences to add; full Bash-like quoting and expansion would require substantially more parser logic.

4. Part 3. Who Expands *? Inspecting argc and argv

```
$ ./args one two three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
```

```
$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [one.c]
argv[3] = [two.c]
argv[4] = [x y]
```

Figure 3.1 – Actual argv produced through Bash

```
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]

$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
```

Figure 3.2 – Actual argv produced through the xv6 shell

4.1. Observe answers

1. Bash produced five arguments for `*.c "x y"`: program name, three expanded filenames, and one quoted argument. xv6 produced four: program name, literal `*.c`, and two words containing the quote characters.

2. The shell, not the target program or kernel, constructs the argument vector. Bash performs the expansions; xv6 performs far fewer transformations.

3. Repeated spaces do not produce empty arguments because both shells tokenize on whitespace.

5. Part 4. Reading the xv6 Shell Source

```
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 4.1 – Actual command kinds found in user/sh.c

```
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
```

Figure 4.2 – Actual execution switch locations in user/sh.c

5.1. Observe answers

1. The five command kinds are `EXEC`, `REDIR`, `PIPE`, `LIST`, and `BACK`. Parentheses group a command but do not create a sixth command type.
2. `cd` is handled by the shell's main process because changing directory in a child would not change the parent shell's working directory.
3. There is no Bash-style globbing or variable-substitution implementation in `sh.c`, which explains the literal `*` and `$HOME` behavior.

6. Conclusion

The comparison showed that xv6 implements the core Unix shell model but intentionally omits many interactive and expansion features provided by Bash. Printing `argv` demonstrated that most apparent "command" behavior is actually shell preprocessing performed before `exec()`.